

数据库系统原理期末总结

第一章：引言 (Chapter 1: Introduction)

1. 文件处理系统的缺点 (Drawbacks of file-processing system)

- **数据冗余和不一致 (Data redundancy and inconsistency):**
 - **解释:** 在传统文件系统中，信息可能分散在多个独立的文件中。同一个数据可能在不同的地方被多次存储，这就造成了数据冗余。例如，一个学生的地址信息可能同时存在于“教务处文件”和“宿管中心文件”中。当学生地址变更时，如果只更新了一个文件而忘记更新另一个，就会导致数据不一致。
- **数据访问困难 (Difficulty in accessing data):**
 - **解释:** 当需要检索满足特定条件的数据时（例如，查询所有居住在“徐汇区”且平均分高于“90分”的学生），需要编写一个全新的应用程序来读取和处理相应的文件。没有一个统一、便捷的查询接口。
- **数据孤立 (Data isolation):**
 - **解释:** 数据被分散存储在不同格式、不同位置的文件中，这些文件之间缺乏关联，形成了“数据孤岛”。要将这些孤立的数据整合起来进行分析变得非常困难。
- **完整性问题 (Integrity problems):**
 - **解释:** 数据的完整性指的是数据的正确性、有效性和相容性。例如，学生的学分总数必须是正数，或者银行账户的余额不能为负数。在文件系统中，这些约束条件需要硬编码到每个数据处理程序中，难以统一维护和修改。
- **原子性问题 (Atomicity of updates):**
 - **解释:** 原子性要求一个操作序列（事务）要么全部执行成功，要么全部不执行。例如，在银行转账操作中，“A 账户扣款”和“B 账户收款”这两个步骤必须捆绑在一起。如果扣款后系统崩溃，导致收款失败，那么整个系统就处于一个不一致的错误状态。文件系统很难保证这种原子性。
- **多用户并发访问 (Concurrent access by multiple users):**
 - **解释:** 当多个用户同时访问和修改同一个数据时，可能会产生冲突和错误。例如，两个票务员同时售卖最后一张火车票。文件系统缺乏有效的并发控制机制来协调这类冲突，可能导致数据混乱（超卖）。
- **安全问题 (Security problems):**

- **解释:** 控制不同用户对不同数据的访问权限在文件系统中很难实现。例如，可能无法轻易地做到“允许学生查看自己的成绩，但不能查看别人的成绩，而教务员可以查看所有学生的成绩”。

2. 核心概念 (Core Concepts)

- **数据库 (Database, DB):**

- **解释:** 数据库是一个长期存储在计算机内、有组织的、可共享的大量数据的集合。它按照特定的数据模型组织、描述和储存，具有较小的冗余度和较高的数据独立性。

- **数据库管理系统 (Database Management System, DBMS):**

- **解释:** DBMS 是位于用户和操作系统之间的一层数据管理软件。它用于科学地组织和存储数据、高效地获取和维护数据。DBMS 是数据库系统的核心，它负责数据库的全部管理工作，包括定义数据结构、提供数据操纵和查询的工具、保证数据安全性和完整性、以及处理并发控制和故障恢复等。例如，MySQL, Oracle, SQL Server 都是 DBMS。

- **数据库系统 (Database System):**

- **解释:** 这是一个更广泛的概念，它通常包括数据库 (DB)、数据库管理系统 (DBMS)、应用程序以及数据库管理员 (DBA) 和用户。

3. 数据抽象 (Abstraction for database)

为了让用户不必关心底层复杂的存储细节，数据库系统提供了不同层次的数据抽象。

- **物理层 (Physical level):**

- **解释:** 这是最低层次的抽象，描述数据在存储设备上**如何被真正存储**。它涉及复杂的数据结构和文件组织方式，如 B+树、哈希索引等。这是由数据库开发者和 DBA 关心的层面。

- **逻辑层 (Logical level):**

- **解释:** 逻辑层描述了数据库中**存储了什么数据**以及这些数据之间存在**什么样的关系**。它对整个数据库的结构进行描述，屏蔽了物理存储的细节。程序员和 DBA 通常在这个层面上工作，例如设计表 (Table) 的结构。

- **视图层 (View level):**

- **解释:** 这是最高层次的抽象，它只描述整个数据库的**某个部分**。视图层是为了简化用户交互，并提供一定程度的安全性而设计的。它允许用户只看到他们关心或被授权看到的数据。例如，一个学生登录教务系统，只能看到他自己的选课和成绩信息，这就是一个视图。

4. 模式与实例 (Schema vs. instance)

- **模式 (Schema):**

- **解释:** 模式是数据库的**逻辑设计或结构描述**，它定义了数据库的框架，包括数据项的名称、类型、关系以及约束等。模式是相对**稳定的**。例如，学生表的模式可以定义为 Student(学号: 字符串, 姓名: 字符串, 年龄: 整数)。

- **实例 (Instance):**

- **解释:** 实例是在某一特定时刻，数据库中存储的**具体数据**的集合。实例是会随着数据的更新而**不断变化的**。例如，在某个时刻，学生表中有三条记录：(S001, 张三, 20), (S002, 李四, 21), (S003, 王五, 19)，这三条记录的集合就是学生关系的一个实例。

5. 数据库用户 (DB users)

- **不同用户类型 (Different user types):** 包括应用程序员、专业用户（使用查询语言）、普通用户（使用图形界面）、DBA 等。
- **数据库管理员的职责 (The duties of DBA):** DBA 是负责管理整个数据库系统的专业人员，其职责包括：定义模式、定义存储结构和存取方法、授予用户权限、指定完整性约束、进行备份和恢复等。

第二章：关系模型 (Chapter 2: Relational Model)

1. 核心概念 (Concepts)

- **关系模式 (Relational schema):** 对关系的描述，包括关系名、组成该关系的属性名集合以及属性域。例如 Student(ID: string, Name: string, Age: int)。
- **关系 (Relation):** 在关系模型中，所有数据都被组织成二维表格的形式，这个二维表就称为关系。
- **关系实例 (Relation instance):** 关系在某一时刻的具体内容，即表中的行和列组成的数据集合。
- **元组 (Tuple):** 表中的一行数据就是一个元组。
- **属性 (Attribute):** 表中的一列数据就是一个属性，每个属性都有一个名字。
- **键 (Key):**
 - **超键 (Superkey):** 一个或多个属性的集合，这个集合可以**唯一标识**表中的一个元组。
 - **候选键 (Candidate key):** 是“最小”的超键，即从该超键中移除任何一个属性，它就不再是超键。
 - **主键 (Primary key):** 从多个候选键中**人为选定一个**，用作该关系的主要标识符。主键的值不能为空 (NOT NULL) 。

- **外键 (Foreign key):** 一个关系中的某个属性（或属性组），它的值是另一个关系的主键。外键用于建立两个关系之间的联系。包含外键的表称为**参照关系 (referencing relation)**，被外键引用的表称为**被参照关系 (referenced relation)**。
- **查询语言 (Query language):** 用于从数据库中请求信息的语言。
 - **过程化 (procedural):** 用户需明确指出查询的**步骤**（怎么做）。关系代数是过程化的。
 - **非过程化 (non-procedural):** 用户只需描述**想要什么**，而无需关心具体步骤（做什么）。SQL 是典型的非过程化语言。

2. 关系代数 (Relational Algebra)

关系代数是一套基于集合论的运算，其操作对象是关系，操作结果也是关系。

- **六个基本算子 (Six basic operators):**
 - **选择 (select, σ):** 从关系中选出满足给定条件的**行**（元组）。
 - **投影 (project, π):** 从关系中选出指定的**列**（属性）。
 - **并 (union, $\cup$):** 合并两个具有相同结构的关系，并去除重复的元组。
 - **差 (set difference, $-$):** 在第一个关系中，去除同时存在于第二个关系中的元组。
 - **笛卡尔积 (cartesian product, $\times$):** 将两个关系的每一个元组合并成一个新的、更长的元组，生成所有可能的组合。
 - **重命名 (rename, ρ):** 改变关系或其属性的名称。
- **其他派生运算 (Other operators):**
 - **交 (Set intersection, $\cap$):** 选出同时存在于两个关系中的元组。
 - **自然连接 (Natural join, $\bowtie$):** 在两个关系的所有同名属性上进行等值连接，并去除重复的列。
 - **赋值 (Assignment, $\leftarrow$):** 将查询结果保存在一个临时关系变量中。
 - **广义投影 (Generalized projection):** 在投影时允许使用算术表达式。
 - **外连接 (Outer join):** 在连接操作的基础上，保留那些在另一个关系中没有匹配项的元组，并用 NULL 填充缺失的属性。分为左外连接、右外连接和全外连接。
 - **聚集函数 (Aggregate functions):** 对一组值进行计算，返回单个值，如求平均 (AVG)、计数 (COUNT)、求和 (SUM) 等。

3. 空值 (Null values)

- **解释:** NULL 是一个特殊的值，表示“未知的”或“不适用的”，它不等于 0 或空字符串。NULL 的存在会使逻辑运算变得复杂（产生“真”、“假”、“未知”三种结果）。

第三、四、五章：SQL (Chapters 3, 4 & 5: SQL)

SQL (Structured Query Language) 是关系数据库的标准语言。

1. SQL 的组成部分 (Components of SQL)

- **数据定义语言 (DDL - Data Definition Language):** 用于定义和管理数据库对象（如表、视图、索引）。
 - CREATE: 创建数据库对象。
 - ALTER: 修改数据库对象的结构。
 - DROP: 删除数据库对象。
- **数据操纵语言 (DML - Data Manipulation Language):** 用于查询和修改数据。
 - SELECT: 查询数据。
 - INSERT: 插入数据。
 - UPDATE: 更新数据。
 - DELETE: 删除数据。

2. 基本操作 (Basic Operations)

- **创建表 (Create table construct):** 使用 CREATE TABLE 语句定义新表，包括表名、列名、数据类型和约束。
- **完整性约束 (Integrity constraints):**
 - NOT NULL: 确保列的值不为空。
 - PRIMARY KEY: 定义主键。
 - FOREIGN KEY: 定义外键，建立表间引用关系。
 - UNIQUE: 确保列中的所有值都是唯一的。
 - CHECK: 规定列值必须满足的条件。
- **基本查询 (Basic Query):**
 - SELECT ... FROM ... WHERE ...: SQL 查询的核心结构。
 - AS (rename): 为列或表提供别名。
 - ORDER BY: 对结果集进行排序。
 - DISTINCT (duplicates): 去除结果集中的重复行。
 - Set operations: UNION, INTERSECT, EXCEPT, 分别对应并、交、差运算。
 - Aggregation: 使用 GROUP BY 子句和聚集函数 (e.g., COUNT, SUM, AVG, MAX, MIN) 对

数据进行分组和汇总。HAVING 子句用于对分组后的结果进行过滤。

3. 高级查询与修改 (Advanced Query & Modification)

- **空值 (Null values):** 使用 IS NULL 或 IS NOT NULL 来判断值是否为空。
- **嵌套子查询 (Nested subqueries):** 一个 SELECT 语句嵌套在另一个语句 (如 SELECT, INSERT, UPDATE, DELETE) 的 WHERE 或 HAVING 子句中。
- **集合比较 (Set comparison):** 使用 ANY, ALL, IN, EXISTS 等关键字来处理子查询返回的集合。
- **视图 (Views):** 视图是基于 SQL 查询结果的虚拟表。它可以简化复杂查询、提供数据独立性、实现安全控制。
- **数据库修改 (Modification of the Database):**
 - DELETE: 删除满足条件的行。
 - INSERT: 插入新行。
 - UPDATE: 修改满足条件的行中的数据。

4. 完整性与安全 (Integrity & Security)

- **断言 (Assertion):** 一个更通用的约束，可以涉及多个表，是数据库需要始终满足的谓词。
- **安全和授权 (Security and authorization):**
 - **授权形式:** 数据库系统通过授权来控制用户对数据的访问。
 - **授权授予 (Granting privileges):** 使用 GRANT 命令将特定操作 (如 SELECT, INSERT, UPDATE) 在特定对象 (如表) 上的权限赋予用户或角色。
 - **角色 (Role):** 权限的集合。可以将权限授予一个角色，再将这个角色授予多个用户，简化权限管理。
 - REVOKE 命令用于收回权限。
- **审计追踪 (Audit trails):** 记录数据库上发生的所有操作，用于事后审计和分析。
- **加密 (Encryption):** 对敏感数据进行加密存储，即使数据被窃取也无法读取。
- **认证 (Authentication):** 验证用户身份的过程，常用方法是密码验证。

第六章：数据库设计与 E-R 模型 (Chapter 6: Database Design and E-R Model)

1. 数据库设计流程 (Database Design Process)

1. **需求分析:** 了解用户的数据需求和功能需求。
2. **概念设计 (Conceptual design):** 使用 E-R (实体-关系) 模型来描述现实世界中的数据对象及其关系，形成与具体 DBMS 无关的概念模型。

3. **逻辑设计 (Logical design):** 将 E-R 模型映射成特定数据模型（如关系模型）的模式。
4. **物理设计 (Physical design):** 确定数据库在物理存储设备上的存储结构和存取方法。

2. E-R 模型核心概念 (E-R Model Concepts)

- **实体 (Entity):** 现实世界中可区分的对象，如一个学生、一门课程。
- **实体集 (Entity set):** 具有相同属性的实体的集合，如所有学生的集合。
- **属性 (Attribute):** 实体所具有的特征。
- **关系 (Relationship):** 多个实体之间的相互关联。
- **关系集 (Relationship set):** 相同类型关系的集合。
- **映射基数 (Degree of mapping cardinality):** 描述参与关系的实体集之间能有多少个实体相关联。常见类型有一对一(1:1)、一对多(1:N)、多对多(M:N)。

3. E-R 设计问题 (ER design issues)

- **实体集 vs. 属性 (use of entity sets vs. attributes):** 某个概念应该设计成一个独立的实体集还是作为另一个实体集的属性？例如，“地址”可以作为学生的属性，也可以设计成一个独立的“地址”实体集。
- **实体集 vs. 关系集 (use of entity sets vs. relationship sets):** 某个概念应该设计成实体集还是关系集？
- **二元 vs. 多元关系 (binary vs. n-ary relationship sets):** 一个关系应该涉及两个实体集还是多个？
- **特殊化/泛化 (specialization/generalization):** 使用 "is a" 的层次结构来表示实体集之间的继承关系。例如，“研究生”是“学生”的一种特殊化类型。

4. E-R 图到关系表的转换 (Reduction to relational tables)

- **解释:** E-R 图作为概念设计的成果，需要被转换成关系数据库的表结构才能被 DBMS 实现。转换有一套标准的规则：
 1. 每个实体集都转换为一个表，实体的属性成为表的列。
 2. 对于关系集，其转换方式取决于映射基数：
 - **多对多:** 为关系集创建一个新表，该表的主键由参与关系的所有实体集的主键组合而成，关系的属性也成为该表的列。
 - **一对多:** 在“多”那方的实体集对应的表中，添加一个外键，该外键引用“一”那方的主键。

- **一对一**: 可以在任意一方的表中添加外键。

第七章：关系数据库设计 (Chapter 7: Relational Database Design)

本章关注如何设计出结构良好、冗余少的关系模式，核心理论是**范式理论 (Normalization Theory)**。

1. 函数依赖 (Functional Dependency, FD)

- **定义**: 在一个关系 R 中，如果对于属性（或属性组）X 的每一个特定值，属性（或属性组）Y 的值都是唯一确定的，那么我们称“Y 函数依赖于 X”，记作 $X \rightarrow Y$ 。例如，在一个学生表中，学号 $\rightarrow$ 姓名，因为一个学号唯一确定一个姓名。
- **函数依赖闭包 (Closure of functional dependency, F⁺)**: 由一个给定的 FD 集合 F，所能逻辑蕴含出的**所有** FD 的集合。
- **Armstrong 公理 (Armstrong's axioms)**: 用于从已知的 FD 推导出新的 FD 的基本规则。
 - **自反律 (reflexivity)**: 如果 Y 是 X 的子集，则 $X \rightarrow Y$ 。
 - **增广律 (augmentation)**: 如果 $X \rightarrow Y$ ，则对于任意 Z，有 $XZ \rightarrow YZ$ 。
 - **传递律 (transitivity)**: 如果 $X \rightarrow Y$ 且 $Y \rightarrow Z$ ，则 $X \rightarrow Z$ 。
- **属性闭包 (Closure of attribute set, X⁺)**: 由属性集 X 和 FD 集 F 共同决定的、可以由 X 函数确定的所有属性的集合。属性闭包常用于判断一个属性集是否为超键。
- **正则覆盖 (Canonical cover)**: 是与原 FD 集等价的、“最简化”的 FD 集合，没有冗余的依赖和冗余的属性。
- **无关属性 (Extraneous attribute)**: 在一个 FD 中，某个可以被移除而不改变 FD 集闭包的属性。

2. 分解 (Decomposition)

- **解释**: 将一个大的、结构不良的关系模式分解成多个小的、结构良好的关系模式的过程。好的分解需要满足两个特性：
 - **无损连接分解 (Lossless-join decomposition)**: 分解后的表通过自然连接操作，可以精确地恢复出原始表的所有信息，不会产生额外的、虚假的元组，也不会丢失原有的元组。
 - **依赖保持分解 (Dependency preservation)**: 原始关系上的所有函数依赖，在分解后的各个表中仍然能够被体现和强制执行。

3. 范式 (Normalization & Normal Forms)

范式是衡量关系模式“好坏”的等级标准。级别越高的范式，其数据冗余越少，更新异常的可能性也越小。

- **第一范式 (1NF):** 关系中的所有属性值都是**原子性的**，不可再分。这是关系模型的基本要求。
- **第二范式 (2NF):** 在满足 1NF 的基础上，消除**非主属性对候选键的部分函数依赖**。
- **第三范式 (3NF):** 在满足 2NF 的基础上，消除**非主属性对候选键的传递函数依赖**。
- **BC 范式 (Boyce-Codd Normal Form, BCNF):** 在满足 3NF 的基础上，消除**主属性对候选键的部分和传递函数依赖**。简单来说，对于模式 R 中任意一个非平凡的函数依赖 $X \rightarrow Y$ ，X 都必须是 R 的一个超键。BCNF 是比 3NF 更强的范式。

第十二、十三章：存储与文件结构 (Chapters 12-13: Storage and File Structure)

1. 物理存储介质 (Physical storage media)

- **分类:**
 - **易失性 (volatile):** 断电后数据丢失，如高速缓存 (Cache)、主存 (Main memory)。速度快，成本高。
 - **非易失性 (non-volatile):** 断电后数据保留，如闪存 (Flash)、磁盘 (Magnetic disk)、光盘 (Optical storage)、磁带 (Tape storage)。速度慢，成本低。

2. 磁盘性能 (Disk performance)

- **关键指标:**
 - **存取时间 (Access time):** = 寻道时间 (seek time) + 旋转延迟 (rotational latency)。
 - **数据传输率 (Data transfer rate):** 从磁盘读出或写入数据的速度。
- **优化:**
 - **磁盘臂调度算法 (Disk-arm-scheduling):** 通过优化磁盘读写头的移动顺序（如电梯算法）来减少总的寻道时间。
 - **块 (Block):** 磁盘与内存之间数据传输的单位。

第十四章：索引 (Chapter 14: Indexing)

1. 基本概念 (Basic concepts)

- **索引 (Index):** 一种特殊的数据结构，旨在**加速**数据库表中数据的检索速度。它类似于书的目录，允许数据库系统不必扫描整张表就能找到所需的数据。
- **搜索码 (Search key):** 用于在索引中查找记录的属性或属性集。

- **主索引/聚簇索引 (Primary index / Clustering index):** 索引的顺序与数据文件中记录的物理顺序**一致**。一张表最多只能有一个聚簇索引。
- **辅助索引/非聚簇索引 (Secondary index / Non-clustering index):** 索引的顺序与数据文件的物理顺序**不一致**。
- **稠密索引 (Dense index):** 为数据文件中的**每一个**搜索码值都创建一个索引项。
- **稀疏索引 (Sparse index):** 只为数据文件中的**一部分**搜索码值创建索引项（通常是每个数据块的第一个值）。

2. 有序索引 (Ordered index)

- **B+树索引 (B+-tree index):**
 - **解释:** 是数据库中最常用的一种平衡多路搜索树。它的特点是：
 1. 所有数据都存储在叶子节点。
 2. 所有叶子节点通过指针连接成一个双向链表，便于范围查询。
 3. 非叶子节点只存储键值（作为路标），不存储数据。
 - B+树能保持动态平衡，使得插入、删除、查询操作的时间复杂度都能维持在对数级别。

3. 哈希索引 (Hash index)

- **解释:** 基于哈希函数建立的索引，将搜索码通过哈希函数计算得到一个桶（bucket）的地址，然后直接访问该桶来查找数据。
- **静态哈希 (Static hashing):** 桶的数量是固定的，可能导致溢出问题。
- **有序索引 vs. 哈希索引:**
 - 哈希索引在**等值查询**（如 `name = '张三'`）上性能极高（接近 $O(1)$ ）。
 - 有序索引（如 B+树）不仅支持等值查询，还非常高效地支持**范围查询**（如 `age > 20`）。

第十五章：查询处理 (Chapter 15: Query Processing)

1. 基本步骤 (Basic steps)

1. **解析和翻译 (Parsing and translation):** 检查 SQL 查询的语法，并将其翻译成内部表示形式（通常是关系代数表达式）。
2. **优化 (Optimization):** 查询优化器选择一个最高效的执行计划。
3. **求值 (Evaluation):** 查询执行引擎根据选定的计划执行查询，并返回结果。

2. 查询代价度量 (Measures of query cost)

- **主要成本:** I/O 成本（磁盘访问）通常是查询成本的主要部分。
- **度量指标:** 磁盘寻道次数 (Seek time)、数据传输块数 (Number of data transfers)。

3. 操作算法 (Operation Algorithms)

- **选择操作 (Selection):**
 - **文件扫描 (File scan):** 顺序扫描整个文件。
 - **索引扫描 (Index scan):** 利用索引快速定位数据。
- **排序操作 (Sorting):**
 - **外排序-归并 (External sort-merge):** 用于处理无法一次性装入内存的大数据量的排序。先将数据分块读入内存排序，写回磁盘，最后将所有有序的子文件归并成一个最终的有序文件。
- **连接操作 (Join):**
 - **嵌套循环连接 (Nested-loop join):** 最简单但可能最低效的连接方法。
 - **块嵌套循环连接 (Block nested-loop join):** 对基本嵌套循环的改进，一次处理一整块数据。
 - **索引嵌套循环连接 (Indexed nested-loop join):** 利用内层关系上的索引来加速匹配。

4. 表达式求值 (Evaluation of expressions)

- **物化 (Materialization):**
 - **解释:** 每一步关系代数运算的结果都被完整地计算出来，并存储在一个临时表中，然后作为下一步运算的输入。这种方式实现简单，但会产生大量的中间结果和 I/O 开销。
- **流水线 (Pipelining):**
 - **解释:** 上一步操作的结果元组，在没有完全生成之前，就立刻被传递给下一步操作进行处理。这避免了读写中间文件的开销，大大提高了效率。

第十六章：查询优化 (Chapter 16: Query Optimization)

1. 优化器核心任务

- **解释:** 对于一个给定的 SQL 查询，可能有多种执行方式（执行计划）。查询优化的目标就是从这些等价的执行计划中，找出**成本最低**的一个。

2. 优化方法 (Optimization Approaches)

- **等价表达式 (Equivalent expressions):** 利用关系代数的等价规则（如连接的交换律和结合律）

来生成不同的逻辑查询计划。

- **代价估算 (Measures of cost):** 优化器需要估算每个操作（如选择、连接）的成本。这依赖于数据库的统计信息，如表的大小、选择基数、连接大小、不同值的数量等。
- **评估计划 (Evaluation plan):** 一个完整的执行计划，详细说明了执行查询所使用的算法和操作顺序。
- **两种主要策略:**
 1. **基于成本的优化 (Cost-based optimization):** 搜索所有可能的执行计划，使用代价模型估算每个计划的成本，并选择成本最低的那个。常用技术是**动态规划 (dynamic programming)**。
 2. **基于启发式的优化 (Heuristic-based optimization):** 使用一组启发式规则来减少搜索空间，例如“总是先执行选择和投影操作”。

第十七章：事务 (Chapter 17: Transactions)

1. 事务 (Transaction)

- **定义:** 事务是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。
- **需要处理的两个主要问题:** 硬件故障和系统崩溃、多事务并发执行。
- **ACID 特性 (Properties: ACID):**
 - **原子性 (Atomicity):** 事务是原子的，要么全部成功，要么全部失败回滚。
 - **一致性 (Consistency):** 事务执行前后，数据库都必须处于一致性状态。
 - **隔离性 (Isolation):** 并发执行的多个事务之间应相互隔离，一个事务的执行不应被其他事务干扰。
 - **持久性 (Durability):** 一旦事务成功提交，它对数据库的修改就是永久性的。

2. 可串行性 (Serializability)

- **并发执行 (Concurrent execution):** 多个事务的指令交错执行。
- **可串行性:** 判断一个并发调度是否“正确”的标准。如果一个并发调度的执行结果与**某个**串行调度（即事务一个接一个地执行）的结果完全相同，那么这个并发调度就是**可串行化**的。
- **冲突可串行性 (Conflict serializability):** 通过分析不属于同一事务的、对同一数据项的**冲突操作**（读-写，写-读，写-写）的顺序来判断。如果一个调度的冲突操作顺序与某个串行调度的冲突操作顺序兼容，它就是冲突可串行化的。

- **视图可串行性 (View serializability):** 一个更宽泛但更难判断的可串行性标准。
- **可恢复性 (Recoverability):** 确保事务的提交顺序。如果事务 T_j 读取了 T_i 写入的数据, 那么 T_i 必须比 T_j 先提交。
 - **级联回滚 (Cascading rollback):** 一个事务的失败导致其他依赖它的事务也必须回滚。
 - **无级联调度 (Cascadeless schedule):** 事务只能读取已提交的事务所写的数据, 从而避免级联回滚。

3. 可串行性测试 (Testing for Serializability)

- **优先图 (Precedence graph):** 用于测试冲突可串行性。图的节点是事务, 如果调度中存在从 T_i 到 T_j 的冲突, 就画一条从 T_i 到 T_j 的有向边。如果该图**无环**, 则调度是冲突可串行化的。

第十八章: 并发控制 (Chapter 18: Concurrency Control)

1. 基于锁的协议 (Lock-based protocols)

- **锁 (Lock):** 一种控制对数据项访问的机制。分为共享锁 (S 锁, 读锁) 和排他锁 (X 锁, 写锁)。
- **锁兼容矩阵 (Lock-compatibility matrix):** 定义了不同类型的锁是否可以被多个事务同时持有一个数据项。
- **死锁 (Deadlock):** 两个或多个事务循环等待对方释放锁。
- **饿死 (Starvation):** 一个事务无限期地等待获取锁。
- **两阶段封锁协议 (Two-phase locking protocol, 2PL):**
 - **核心思想:** 事务的加锁和解锁分为两个阶段。
 1. **增长阶段 (Growing Phase):** 事务可以获得锁, 但不能释放锁。
 2. **缩减阶段 (Shrinking Phase):** 事务可以释放锁, 但不能再获得新锁。
 - **作用:** 2PL 协议可以**保证**冲突可串行性, 但**可能**发生死锁。
 - **严格两阶段封锁 (Strict 2PL):** 要求事务持有的所有**写锁**必须在事务提交后才能释放。可以避免级联回滚。
 - **强两阶段封锁 (Rigorous 2PL):** 要求事务持有的**所有锁** (读锁和写锁) 都必须在事务提交后才能释放。

2. 基于时间戳的协议 (Timestamp-based protocols)

- **解释:** 为每个事务分配一个唯一的、单调递增的时间戳。当发生冲突时, 通过比较事务的时间戳来决定操作的执行顺序, 从而保证可串行性, 且不会发生死锁。

4. 死锁处理 (Deadlock handling)

- **死锁预防 (Deadlock prevention):** 通过协议来确保系统中永远不会发生死锁。例如，要求所有事务按相同顺序获取锁，或者使用 wait-die (非抢占) 和 wound-wait (抢占) 方案。
- **死锁检测 (Deadlock detection):** 允许死锁发生，但系统会周期性地检测死锁。
 - **等待图 (Wait-for graph):** 用于死锁检测。节点是事务，如果事务 T_i 在等待 T_j 持有的锁，就画一条从 T_i 到 T_j 的边。如果图中存在**环路**，就表示发生了死锁。检测到死锁后，系统会选择一个“牺牲品”事务进行回滚，打破环路。

第十九章：恢复系统 (Chapter 19: Recovery System)

1. 恢复方法 (Approaches for recovery)

- **基于日志的恢复 (Log-based recovery):**
 - **解释:** 系统维护一个称为**日志 (log)** 的文件，记录所有对数据库的更新操作。当发生故障时，DBMS 的恢复管理器可以根据日志信息来撤销 (UNDO) 未完成的事务或重做 (REDO) 已提交的事务，使数据库恢复到一致状态。
 - **延迟修改 (Deferred database modification):** 所有修改操作先记录在日志中，直到事务提交时才真正写入磁盘。
 - **立即修改 (Immediate database modification):** 修改操作立即写入磁盘，但同时也会写入日志。
- **检查点 (Checkpoint):**
 - **解释:** 一个系统操作，它将内存中所有已修改的缓冲区数据强制写入磁盘，并在日志中写入一个 <checkpoint> 记录。检查点可以缩短恢复过程所需的时间，因为它减少了需要扫描的日志量。